

```
1  
2  
3  
4  
5 > ./compilar --origem=suaíli --destino=python3  
6  
7  
8
```

Compilador Suaíli → Python 3

Um projeto de tradução léxica, sintática e semântica
para quebrar a barreira do inglês na programação.

Equipe: Rodrigo Balthazar,
Samuel Peixoto, Vitor Reis

Orientador: Eduardo Sidney
da Silva Xavier

Teoria da Computação e Compiladores |
Projeto Final A3 (Unifacs, 2026)



50 Milhões

Falantes nativos de Suaíli no mundo.

Presença Ativa

Idioma oficial e amplamente utilizado na educação e tecnologia da região litoral sul africana.

A Barreira

A programação moderna exige fluência prévia no idioma inglês para estruturação lógica (if, while, print).

A missão deste compilador é provar que a lógica computacional pode ser abstraída e ensinada nativamente, convertendo código Suaíli em Python 3 executável.

O Alicerce Tecnológico

HTML /
CSS / JS

Camada 1

Editor
Interativo

Execução 100% no
navegador (client-side).

JS
ES6

Camada 2

Motor do
Compilador

Hospeda o Lexer, Parser e
Gerador de Código em
JavaScript puro, sem
dependências externas.

Python
3

Camada 3

Linguagem
Alvo

Código validado,
semanticamente correto
e pronto para execução.

A Pedra de Roseta: Tipos de Dados

Suaíli	Python 3	Exemplo Prático
nambari	⑤ int	nambari x = 10
desimali	3.14 float	desimali pi = 3.14
maneno	"..." str	maneno s = "01a"
kweli_uongo	ON OFF bool	kweli_uongo b = kweli

Nota Cultural:

Os literais booleanos foram mapeados para kweli (Verdadeiro / True) e uongo (Falso / False).

Dicionário Operacional

Fluxo Condicional

`kama` → `if`
`sivyo` → `else`

Laços de Repetição

`wakati` → `while`
`fanya ... wakati` → `while True + break`
`kwa` → `for` (mapeado para `while`)

Entrada e Saída (E/S)

`chapisha()` → `print()`
`soma()` → `input()`
`soma_nambari()` → `int(input())`

Operadores Lógicos

`na` → `and`
`au` → `or`
`si` → `not`

As Regras do Jogo: Gramática BNF

```
if_stmt ::= 'kama' '(' expr ')' block  
         ( 'sivyo' ( if_stmt | block ) )?
```

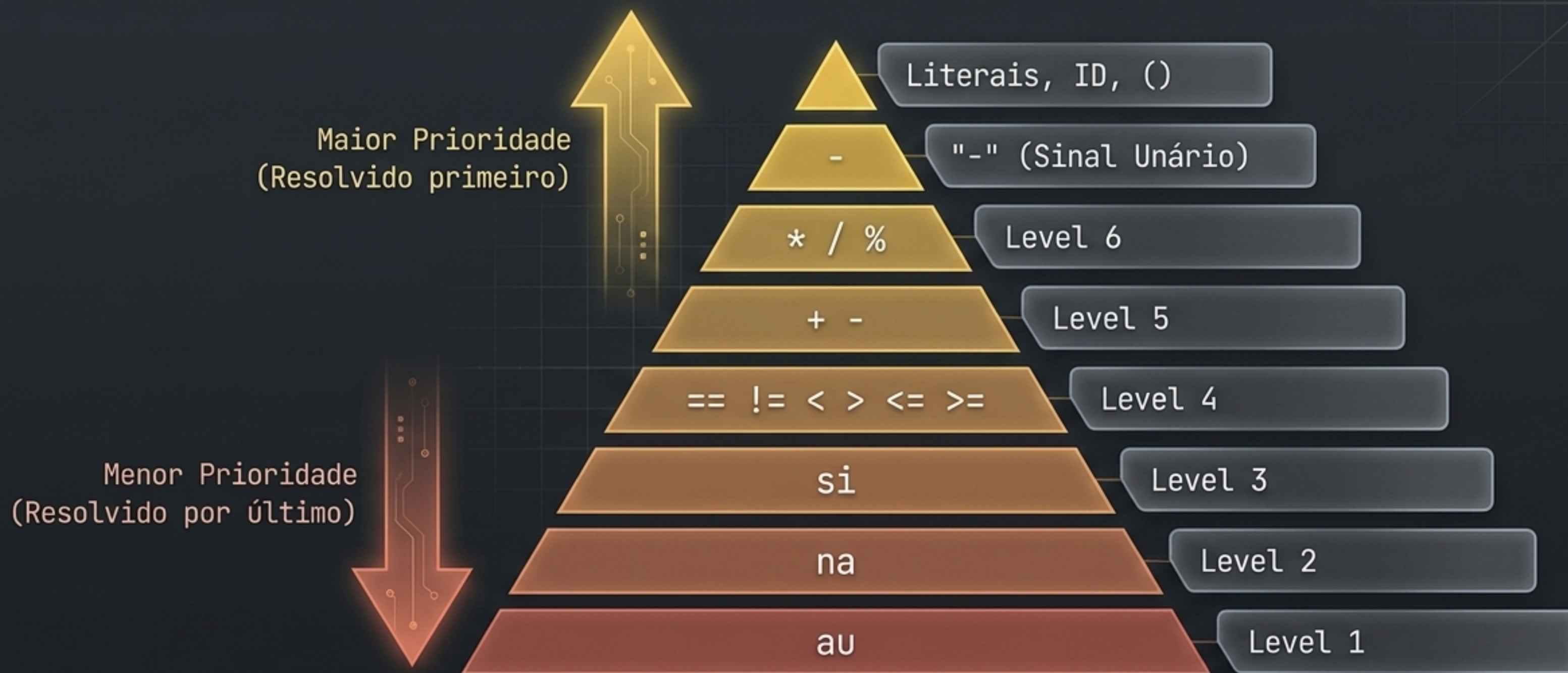
Terminal Obrigatório:
Token exato
reconhecido pelo
Lexer.

Regra Recursiva:
Avalia qualquer
expressão matemática
ou lógica.

Operador Opcional (?):
Resolve a ambiguidade
clássica do
dangling-else.

1. Gramática Livre de Contexto (GLC)
2. Sem recursão à esquerda (Descendente Recursivo)
3. Blocos estritamente delimitados por { }

A Hierarquia Matemática: Precedência



A precedência é garantida pela profundidade hierárquica das regras de produção na gramática, não por força bruta.

A Máquina de Tradução



O Scanner (Análise Léxica)

nambari x = 10

[INT]

[ID: 'x']

[ASSIGN]

[NUM_INT: 10]

O scanner descarta silenciosamente espaços em branco e comentários de linha iniciados por //.

A Árvore (Análise Sintática)



Descendente Recursivo

Cada produção da gramática BNF é mapeada para uma função dedicada que consome os tokens para construir esta estrutura hierárquica.

A Guarda Costeira: Verificações Semânticas



- ✓ **Declaração Obrigatória**
Variáveis não declaradas abortam imediatamente a compilação.
- ✓ **Isolamento de Pilha**
Ao fechar uma chave `}`, o escopo local é destruído na memória.
- ✓ **Checagem de Tipagem Lógica**
Comandos como `soma_nambari()` injetam cast explícito (`int()`) no alvo.

Injeção de Indentação (Geração de Código)

Suaíli - Estrutura por Chaves

```
kwa (nambari i=0; i<5; i=i+1) {  
    chapisha(i)  
}
```



Python 3 - Estrutura por Espaço

```
i = 0  
while i < 5:  
    [.....]print(i)  
    [.....]i = i + 1
```

O gerador suprime as chaves e injeta estritamente 4 espaços matemáticos por nível de profundidade na árvore sintática.

0 Compilador em Ação: Fibonacci

Suaíli Source

```
nambari a = 0
nambari b = 1
wakati (cont > 0) {
    chapisha(a)
    nambari temp = a + b
    a = b
    b = temp
}
```

Python Target

```
a: int = 0
b: int = 1
while cont > 0:
    print(a)
    temp: int = a + b
    a = b
    b = temp
```

Saída do Terminal

```
> 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...
> █
```


À Prova de Falhas: Resiliência

```
nambari x = 10
y = x + 2 // 'y' nunca foi declarado
```

⊗ [COMPILAÇÃO ABORTADA] ERRO: Variável não declarada: 'y'

Categoria	Comportamento Diagnóstico
Semântica	Bloqueia variáveis órfãs e inconsistência de tipos.
Sintaxe	Informa: 'Esperado TIPO X, encontrado TIPO Y'.
Léxica	Aponta o índice exato do caractere desconhecido.

O parser interrompe a tradução no primeiro erro, prevenindo lixo sintático no arquivo Python.

Referências Acadêmicas

AHO, A. V. et al. (2008)

Compiladores: Princípios, Técnicas e Ferramentas (O Livro do Dragão).

LOUDEN, K. C. (2004) & COOPER, K. (2011)

Engenharia de Compiladores e Princípios Práticos.

PYTHON SOFTWARE FOUNDATION (2026)

Documentação Oficial da Linguagem Alvo (Python 3).

INSPIRAÇÃO ORIGINAL

Projeto Pyswahili (PyPI).

Asante

(Obrigado em Suaíli)